

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS 301: Software Engineering

Software Architecture Specification

Capstone Project 2026 - Demo 2

Team Name: OmniTech

July 31, 2026

Contents

1	Introduction	3
2	Project Owner	3
3	Architectural Requirements	4
3.1	Overall Software Architecture	4
3.2	Primary Architectural style	4
3.3	Communication Patterns	4
3.3.1	Request-Response Model	4
3.3.2	Push-Pull Model	4
3.3.3	Event-Driven Messaging	5
3.4	Communication Protocols and Standards	5
3.5	Architectural Patterns Per Subsystem	5
3.5.1	Backend API	5
3.5.2	Android Application	6
3.5.3	Web Dashboard	6
3.6	Design Patterns	7
3.6.1	Observer Pattern	7
3.6.2	Strategy Pattern	7
3.7	Constraints	7
3.7.1	Hardware Constraints	7
3.7.2	Platform Constraints	7
3.7.3	Real-Time Processing Constraints	7
3.7.4	Privacy and Legal Constraints	8
3.7.5	Technology Constraints	8
3.8	High-Level Architecture Diagram	9
3.9	Quality Requirements to Architectural Decisions	9
4	Technology Requirements	10
4.1	TR1 - Mobile Application Runtime:	10
4.2	TR2 - Backend Runtime:	10
4.3	TR3 - Database:	10
4.4	TR4 - Containerization:	10
4.5	TR5 - Cloud Infrastructure:	10
4.6	TR6 - Frontend Runtime:	11
4.7	TR7 - Package Management:	11
4.8	TR8 - Version Control:	11
4.9	TR9 - Android Development Environment:	11
4.10	TR10 - External APIs and Services:	11

4.11	TR11 - Authentication:	11
4.12	TR12 - CI/CD Pipeline:	11
4.13	Deployment Diagram	12
5	API Services Contract	12
5.1	Auth Endpoints	13
5.2	Trips Endpoints	17
5.3	Contacts Endpoints	25
5.4	Badges Endpoints	32
5.5	Leaderboard Endpoints	35
5.6	Vehicle Endpoints	37
5.7	Map Endpoints	41
5.8	Notifications Endpoints	43
5.9	User Devices Endpoints	44
5.10	Health Endpoint	45
5.11	CI pipeline flow diagram	46
5.12	CD pipeline flow diagram	47

1 Introduction

Driving Tracker is an intelligent Android-based driving companion developed by OmniTech. The system aims to improve road safety, fuel efficiency, and vehicle maintenance by combining mobile sensors, GPS tracking, and vehicle diagnostics into a single integrated platform. Unlike existing solutions that focus only on navigation or diagnostics, Driving Tracker provides drivers with real-time feedback on driving behavior and vehicle health.

This document explores the software requirements and design specifications of the Driving Tracker application, as well as the architectural and technology requirements.

2 Project Owner

The project owner and client of the Driving Tracker is Gendac, and our industry mentors are Kevin Cowdrey, Sthabiso Jaffar, and Brandon Craytor.

3 Architectural Requirements

This section defines the architectural requirements for the Driving Tracker system. These requirements govern the structural decisions, quality attributes, design patterns, and constraints that guide development across the application.

3.1 Overall Software Architecture

This section discusses the high-level architecture of the Driving Tracker system across three levels of granularity, covering the primary architectural style, communication patterns between subsystems, and the architectural patterns applied within each subsystem.

3.2 Primary Architectural style

The Driving Tracker system adopts a Client-Server architecture as its primary architectural style. The system consists of two distinct client types: the Android mobile application, the web-based fleet management dashboard (and indirectly, OBD-II Bluetooth adapters as peripheral data sources). These clients interact with a centralised backend server responsible for data persistence, business logic, leaderboard computation, fleet coordination, and notification dispatch. This architecture is appropriate for Driving Tracker because the leaderboard, gamification, fleet management, and trusted contact features inherently require a shared server that multiple clients can communicate with.

3.3 Communication Patterns

Communication patterns used between subsystems:

3.3.1 Request-Response Model

The primary communication pattern between clients and the backend is the request-response model over HTTP. The Android application and the web dashboard send HTTP requests to the REST API, which processes the request and returns a JSON response. This pattern governs all standard operations including authentication, trip submission, leaderboard retrieval, vehicle management, and trusted contact alerts.

3.3.2 Push-Pull Model

A push-pull model is used for data synchronization and alerting. The Android application buffers trip and sensor data locally and pushes it to the backend in the background when connectivity is restored. Trusted contact alerts and safety notifications are pushed from the backend to the Android client via Firebase Cloud Messaging. The web dashboard pulls fleet data from the backend on demand and on a configurable polling interval.

3.3.3 Event-Driven Messaging

A event-driven messaging pattern is used to process sensor and OBD data within the Android application. Raw sensor readings are emitted as events through Kotlin Flow streams. Downstream consumers/modules, including the harsh event detector, and UI layer, subscribe to these streams and react to readings independently. This decouples data production from data consumption and allows the sampling rate to differ from the processing and rendering rate.

3.4 Communication Protocols and Standards

The application-level communication protocols and standards applied across the system are as follows:

- REST over HTTPS is used for all client-to-server API communication. REST was selected over SOAP, GraphQL, and RPC approaches because it is stateless, widely supported across Android and web clients, and sufficient for the CRUD-heavy operations of this system. GraphQL was considered but ruled out given the system does not require dynamic field selection at the client level.
- HTTP/1.1 is the underlying transport protocol. TLS 1.2 or higher is required for all connections, enforced at the server and network layer.
- Bluetooth Classic is used for OBD-II communication between the Android device and the ELM327 adapter. This is more so a hardware constraint imposed by ELM327 adapters.
- JSON is the data interchange format for all REST API responses and requests.

3.5 Architectural Patterns Per Subsystem

The following architectural patterns are applied within each subsystem, with multiple combined where appropriate:

3.5.1 Backend API

- Layered (n-tier) Architecture: The backend is structured into three distinct layers: the routing layer, the controller layer, and the service layer. Each layer has a single responsibility and communicates only with the layer directly below it.
- Service-Oriented Architecture (SOA): Backend functionality is decomposed into independently operable services including authentication, trip management, gamification, fleet management, notifications, and user management. Each service exposes a well-defined interface.

3.5.2 Android Application

- **Modular Architecture:** The Android application is divided into independently developed and tested feature modules covering sensors, OBD communication, trip tracking, gamification, notifications, and data synchronization.
- **Offline-First Architecture:** The application treats the local Room database as the primary data store. All writes are committed locally before being synchronized to the backend, ensuring uninterrupted trip recording in low-connectivity.
- **Event-Driven Architecture:** The sensor pipeline uses reactive event streams implemented with Kotlin Flow, allowing multiple subsystems to consume sensor data independently without tight coupling.

3.5.3 Web Dashboard

- **Component-Based Architecture:** follows The React and Next.js web dashboard a component-based UI architecture, decomposing screens into reusable, independently testable UI components with centralized state management.

3.6 Design Patterns

3.6.1 Observer Pattern

Sensor data streams on Android are modelled as observable flows using Kotlin Flow. UI components and scoring logic subscribe to these flows and react to new readings. This pattern decouples the sensor sampling rate from the UI refresh rate and allows multiple consumers to receive the same sensor events independently.

3.6.2 Strategy Pattern

Driver scoring and harsh event detection could use configurable strategies that encapsulate the scoring algorithm. Different vehicle types or driving profiles may use different scoring strategies without altering the core trip recording logic. This also provides an easier path to introduce ML based scoring while alongside rule-based scoring.

3.7 Constraints

3.7.1 Hardware Constraints

- OBD-II diagnostics require a physical ELM327-compatible Bluetooth adapter connected to the vehicle. Vehicles made before 1996 may not have an OBD-II port and will not support this feature.
- Sensor availability varies between Android devices. The system must implement adaptive thresholds and degrade gracefully on devices that do not have specific sensors.
- Bluetooth Classic is required for ELM327 communication. Although most devices do have Bluetooth support, those without cannot use OBD features.

3.7.2 Platform Constraints

- Background location access requires permission which needs to be explicitly granted by the user on Android 10 and above.
- Background execution is subject to Android battery optimization restrictions. Ideally, the system should use a foreground service with a persistent notification during active trip recording to prevent the system from terminating the process.

3.7.3 Real-Time Processing Constraints

- Road noise, vibration, and environmental interference could produce false positives in harsh event detection. Configurable sensitivity levels, as well as other techniques like smoothing, should be utilised to reduce false alert rates.
- Sensor and OBD data processing must not block the main thread, to avoid delaying UI

or navigation. Processing must occur on background threads or coroutines.

3.7.4 Privacy and Legal Constraints

- Location tracking must not begin until the user has granted location permissions and provided consent through the in-app consent flow.
- Live trip sharing with trusted contacts must be managed per trip. Contacts must not be able to view location data without the user actively sharing their location.

3.7.5 Technology Constraints

- Backend infrastructure must be deployable to Microsoft Azure.
- Background execution is subject to Android battery optimization restrictions. Ideally, the system should use a foreground service with a persistent notification during active trip recording to prevent the system from terminating the process.

3.8 High-Level Architecture Diagram

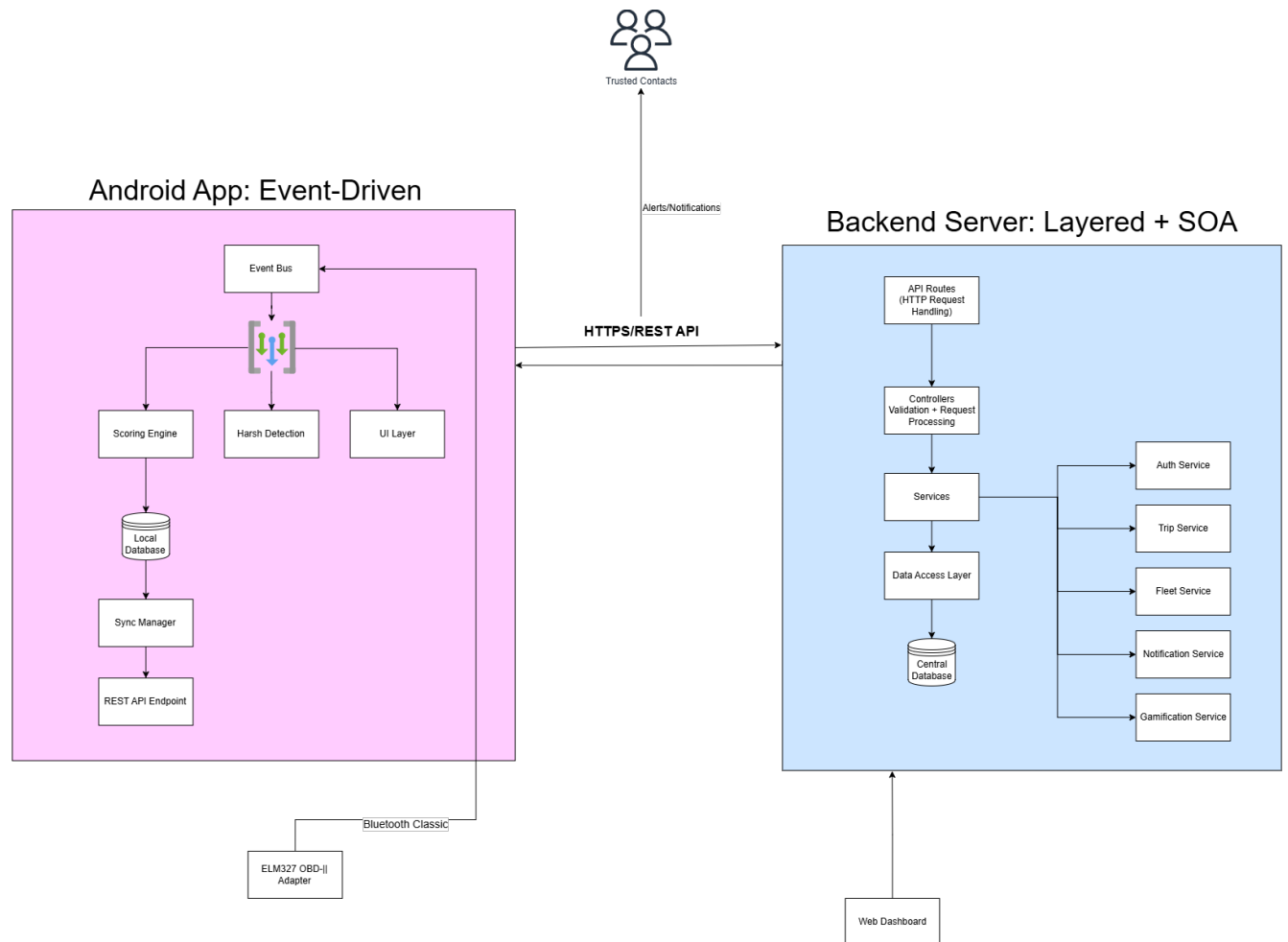


Figure 1: Architectural Diagram

3.9 Quality Requirements to Architectural Decisions

Quality Requirement	Architectural Decision
UI should respond $\leq 200\text{ms}$	Singleton obd manager and sensor fusion manager with Kotlin flow to avoid polling
Sensor and OBD reflect in ≤ 2 seconds	Sensor sampling, OBD polling and Room Db operations execute on background dispatchers via coroutines
Trip data not lost on connectivity loss	Room Db persistence on device for trip data
Maintain testing coverage $\geq 80\%$	Individually testable layers and modules
Availability $\geq 99\%$ uptime	Health check ping every minute (Azure App Services), and rate limiting on endpoints to avoid DOS attacks

Table 1: Mapping Quality Requirements to Architectural Decisions

4 Technology Requirements

4.1 TR1 - Mobile Application Runtime:

Android device running Android 8.0 or higher

4.2 TR2 - Backend Runtime:

Node.js version 20 or higher

4.3 TR3 - Database:

PostgreSQL version 15 or higher

Prisma ORM

4.4 TR4 - Containerization:

Docker Desktop with Docker Compose v2 or higher

4.5 TR5 - Cloud Infrastructure:

Azure app service

Azure DB for PostgreSQL Flexible Server

Azure Container Registry

4.6 TR6 - Frontend Runtime:

Next.js 15

4.7 TR7 - Package Management:

Yarn 4 via Corepack

4.8 TR8 - Version Control:

Git version 2.x or higher

4.9 TR9 - Android Development Environment:

Android Studio

Retrofit (HTTP client for Android and Java)

4.10 TR10 - External APIs and Services:

Azure Maps Web SDK

Bluetooth OBD-||

carapi

Firebase Cloud Messaging

4.11 TR11 - Authentication:

JSON Web Tokens

4.12 TR12 - CI/CD Pipeline:

Github actions

4.13 Deployment Diagram

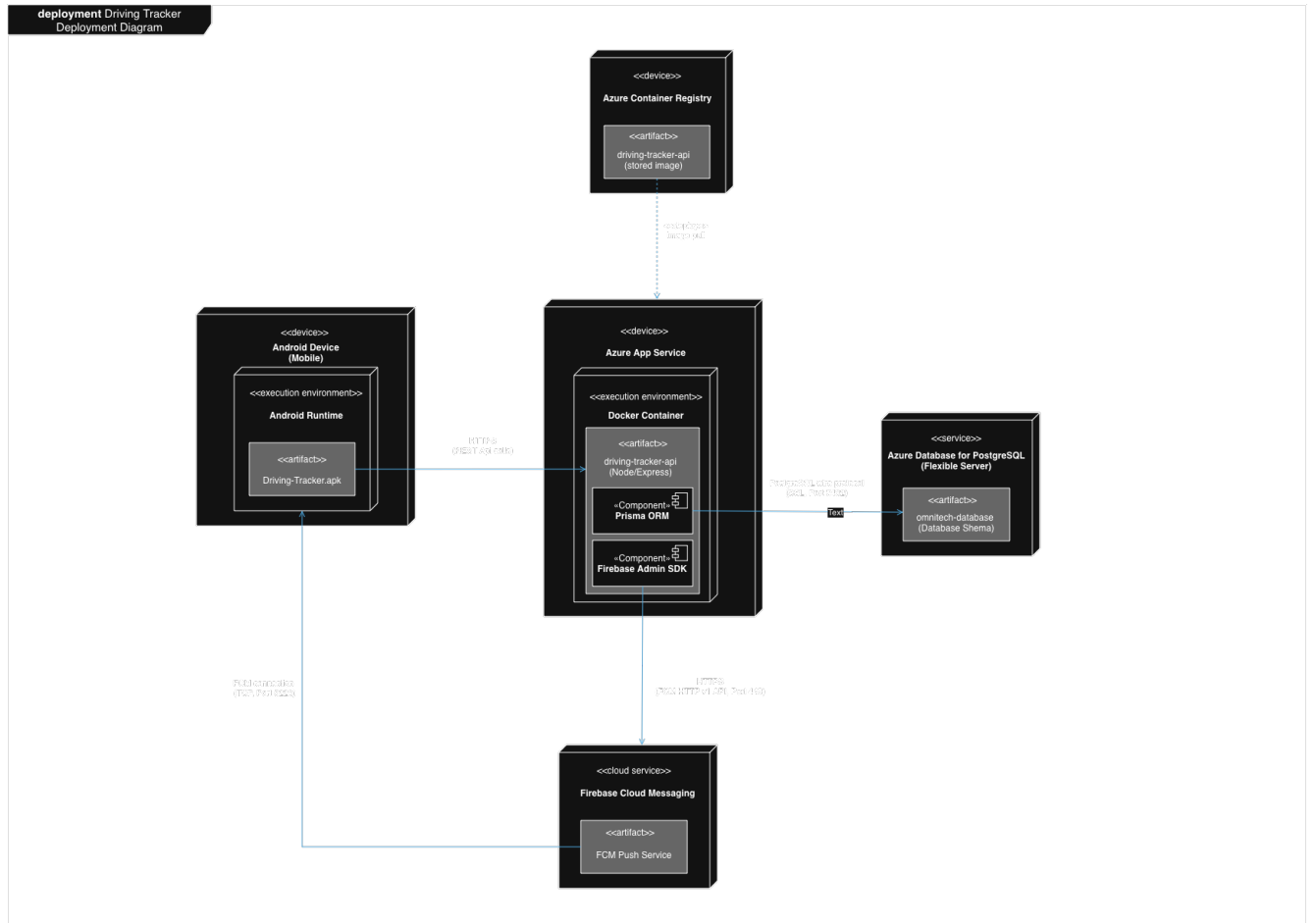


Figure 2: Deployment Diagram (including technologies)

5 API Services Contract

This section defines the REST API service contract for the Driving Tracker backend, covering every endpoint's request/response shape, required headers, and error conditions. It serves as the authoritative interface agreement between the Android application, the web dashboard, and the backend services described in Section ??.

Overview

- **Protocol:** REST
- **Base URL:** *TBD*
- **Authorization:** JWT

All endpoints below other than `/health`, `/api/auth/register`, `/api/auth/login`, and `/api/auth/refresh` require the header:

```
"Authorization": "Bearer <jwt_token>"
```

If this header is missing or the token is invalid/expired, every protected endpoint responds the same way, before reaching its own logic:

Response 401:

```
{
  "error": "UNAUTHORIZED",
  "message": "No token provided"
}
```

(or error: "TOKEN_EXPIRED", message: "Token expired" / message: "Invalid Token")

5.1 Auth Endpoints

POST /api/auth/register

Request Body

```
{
  "email": "String",
  "username": "String",
  "password": "String",
  "name": "String",
  "surname": "String",
  "phone_number": "String",
  "dob": "String",
  "consent_status": "boolean"
}
```

Response 201:

```
{
  "token": "String",
  "refresh_token": "String"
}
```

Response 409:

```
{
  "error": "EMAIL_TAKEN | USERNAME_TAKEN",
  "message": "String"
}
```

Response 422:

```
{
  "error": "INVALID_CONSENT_STATUS | INVALID_USERNAME |
    INVALID_NAME | INVALID_SURNAME | INVALID_PHONE |
    INVALID_EMAIL | INVALID_DOB | INVALID_PASSWORD",
  "message": "String"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

POST /api/auth/login

Request Body

```
{
  "identifier": "email|username",
  "password": "string"
}
```

Response 201:

```
{
  "token": "jwt_token_string",
  "refresh_token": "refresh_jwt"
}
```

Response 401:

```
{
  "error": "INVALID_CREDENTIALS | INVALID_PASSWORD",
  "message": "String"
}
```

Response 429:

```
{
  "error": "TOO_MANY_ATTEMPTS",
  "message": "Too many login attempts, try again later"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

```
}
```

POST /api/auth/logout

Headers

```
{  
  "Authorization": "Bearer <jwt_token>"  
}
```

Response 200:

```
{  
  "message": "Logged out successfully"  
}
```

Response 500:

```
{  
  "error": "INTERNAL_SERVER_ERROR",  
  "message": "Failed to log out"  
}
```

POST /api/auth/refresh

Request Body

```
{  
  "refresh_token": "String"  
}
```

Response 200:

```
{  
  "token": "access_jwt",  
  "refresh_token": "refresh_jwt"  
}
```

Response 400:

```
{  
  "error": "MISSING_REFRESH_TOKEN",  
  "message": "Refresh token required"  
}
```

Response 401:

```
{  
  "error": "UNAUTHORIZED",
```



```
{
  "message": "Invalid refresh token"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

GET /api/auth/profile

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "data": {
    "user_id": "uuid",
    "username": "String",
    "name": "String",
    "surname": "String",
    "email": "String",
    "phone_number": "String",
    "dob": "dateTime",
    "trip_count": "int",
    "badge_count": "int",
    "vehicle_count": "int"
  },
  "message": "Profile retrieved successfully"
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Failed to retrieve profile"
}
```

5.2 Trips Endpoints

POST /trips/start_trip

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{
  "vehicle_id": "uuid",
  "start_date": "dateTime",
  "data_source": "OBD | PHONE",
  "start_location": {
    "lat": "float",
    "lng": "float"
  },
  "end_location": { "lat": "float", "lng": "float" } (optional),
  "share_with_contacts": ["contact_id", ...] (optional)
}
```

Response 200:

```
{
  "message": "trip successfully started",
  "data": {
    "trip_id": "uuid",
    "data_source": "OBD | PHONE",
    "planned_distance_km": "float | null",
    "fuel_estimate": "float | null"
  }
}
```

Response 403:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 403:

```
{
  "error": "USER_NOT_FOUND",
  "message": "Unauthorized"
}
```

Response 409:

```
{
  "error": "Trip already in progress"
}
```

Response 422:

```
{
  "error": "MISSING_REQUIRED_FIELDS",
  "message": "user or vehicle not known"
}
```

POST /trips/:trip_id/readings/record

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{
  "recorded_at": "dateTime",
  "location": {
    "lng": "double",
    "lat": "double"
  },
  "data_source": "OBD | PHONE",
  "speed_kmh": 60.5,
  "accelerometer": 0.12,
  "gyroscope_x": 0.01,
  "gyroscope_y": 0,
  "gyroscope_z": -0.02,
  "rpm": 2200,
  "coolant_temp_c": 88,
  "fuel_trim_percent": 1.5,
  "throttle_position": 22.4,
  "dtc_codes": [
    "P0420"
  ]
}
```

Response 201:

```
{
  "message": "Recorded successfully"
}
```

```
}
```

Response 401:

```
"error": "Fill all valid fields"
```

Response 400:

```
{  
  "error": "UNAUTHORIZED"  
}
```

Response 404:

```
{  
  "error": "TRIP_NOT_FOUND",  
  "message": "Trip not found"  
}
```

Response 500:

```
{  
  "error": "INTERNAL_SERVER_ERROR",  
  "message": "String"  
}
```

PATCH /trips/:trip_id/end_trip

Headers

```
{  
  "Authorization": "Bearer <jwt_token>"  
}
```

Request Body

```
{  
  "end_time": "dateTime",  
  "route_polyline": "string",  
  "distance_km": "float",  
  "duration_minutes": "integer",  
  "fuel_estimate": "float",  
  "status": "COMPLETED | ABORTED",  
  "end_location": "{  
    "lat": "float",  
    "lng": "float"  
  }"  
}
```

Response 200:

```
{
  "message": "Trip completed successfully",
  "data": {
    "trip_id": "uuid",
    "username": "string",
    "status": "COMPLETED",
    "distance_km": "float",
    "duration_minutes": "integer",
    "fuel_estimate": "float",
    "scores": {
      "safety_score": "float",
      "eco_score": "float",
      "overall_score": "float"
    },
    "is_first_trip": "boolean",
    "driverProfile"
  }
}
```

Response 403:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 403:

```
{
  "error": "FORBIDDEN",
  "message": "You do not own this trip"
}
```

Response 404:

```
{
  "error": "TRIP_NOT_FOUND",
  "message": "Trip not found"
}
```

Response 409:

```
{
  "error": "TRIP_ALREADY_COMPLETED"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

GET /trips/:trip_id/summary

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "data": {
    "trip_id": "uuid",
    "vehicle_id": "uuid",
    "started_At": "dateTime",
    "ended_At": "dateTime",
    "status": "string",
    "data_source": "OBD | PHONE | MIXED",
    "route_polyline": "string",
    "distance_km": "float",
    "duration_minutes": "int",
    "fuel_estimate": "float",
    "scores": { "safety_score": "float", "eco_score": "float", "
      overall_score": "float" } | null,
    "dtc_codes": [
      "string"
    ],
    "destination_latitude": "float | null",
    "destination_longitude": "float | null",
    "events": [
      {
        "event_id": "uuid",
        "event_type": "string",
        "longitude": "float",
        "latitude": "float",
        "severity": "float",
        "sensor_source": "string",
        "time_stamp": "dateTime"
      }
    ]
  }
}
```

```
}  
}
```

Response 401:

```
{  
  "error": "UNAUTHORIZED",  
  "message": "Can not view this trip"  
}
```

Response 403:

```
{  
  "error": "FORBIDDEN",  
  "message": "You do not own this trip"  
}
```

Response 404:

```
{  
  "error": "NOT_FOUND",  
  "message": "Trip not found"  
}
```

Response 500:

```
{  
  "error": "Internal server error"  
}
```

GET /trips/history

Headers

```
{  
  "Authorization": "Bearer <jwt_token>"  
}
```

Query Parameters

- start_date = date (defaults to 30 days ago if not provided)
- end_date = date (defaults to now if not provided)
- status = COMPLETED | IN_PROGRESS | ABORTED

Response 200:

```
{
  "username": "string",
  "start_date": "date",
  "end_date": "date",
  "total_trips": "integer",
  "trips": [ <full trip record, including a nested trip_scores[]
    array> ],
  "meta": {
    "mean_distance": "float",
    "mean_minutes": "float"
  }
}
```

Response 400:

```
{
  "error": "UNAUTHORIZED",
  "message": "user not identified"
}
```

Response 404:

```
{
  "error": "User not found"
}
```

Response 400:

```
{
  "error": "Invalid start date | Invalid end date"
}
```

Response 500:

```
{
  "error": "Internal server error"
}
```

POST /trips/:trip_id/events/log

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```


Request Body

```
{
  "event_type": "HARSH_BRAKE | HARSH_ACCELERATION | SHARP_CORNER |
    CRASH_LIKE",
  "location": {
    "lat": "float",
    "lng": "float"
  },
  "severity": "float",
  "sensor_source": "ACCELEROMETER | GYROSCOPE | OBD",
  "timestamp": "dateTime"
}
```

Response 201:

```
{
  "data": {
    "event_id": "uuid",
    "trip_id": "uuid",
    "type": "string",
    "severity": "float",
    "sensor_source": "string",
    "timestamp": "dateTime",
    "message": "Event logged successfully"
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 403:

```
{
  "error": "FORBIDDEN",
  "message": "You do not own this trip"
}
```

Response 404:

```
{
  "error": "TRIP_NOT_FOUND",
  "message": "Trip not found"
}
```

```
}
```

Response 400:

```
{
  "error": "INVALID_EVENT_TYPE"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

5.3 Contacts Endpoints

POST /contacts

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{
  "identifier": "String (username, email, or user_id)"
}
```

Response 201:

```
{
  "data": {
    "contact_id": "uuid",
    "username": "String"
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 400:

```
{
  "error": "CANNOT_ADD_USER",
  "message": "Cannot add this user as a contact"
}
```

Response 404:

```
{
  "error": "USER_NOT_FOUND",
  "message": "User not found"
}
```

Response 409:

```
{
  "error": "ALREADY_TRUSTED_CONTACT",
  "message": "contact already added"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

GET /contacts

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "data": {
    "contacts": [
      {
        "contact_id": "uuid",
        "username": "String",
        "name": "String",
        "email": "String"
      }
    ]
  },
}
```

```
{
  "message": "Contacts successfully retrieved"
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 403:

```
{
  "error": "CANNOT_ACCESS_CONTACTS",
  "message": "Cannot access these contacts"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

POST /contacts/alerts

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{
  "event_type": "String",
  "event_id": "uuid",
  "message": "String (optional)",
  "contacts": [
    {
      "contact_id": "uuid"
    }
  ]
}
```

Response 200:

```
{
  "message": "Contacts successfully alerted"
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 400:

```
{
  "error": "BAD_REQUEST",
  "message": "Invalid request body"
}
```

Response 403:

```
{
  "error": "CANNOT_ACCESS_CONTACTS",
  "message": "Cannot access these contacts"
}
```

Response 404:

```
{
  "error": "CONTACT_NOT_FOUND",
  "message": "Cannot find contact"
}
```

Response 404:

```
{
  "error": "EVENT_NOT_FOUND",
  "message": "Cannot find event"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

POST /contacts/share_location

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{ "trip_id": "uuid", "contacts": [ { "contact_id": "uuid" } ] }

// Also accepts the wrapped shape:
{ "contacts": { "trip_id": "uuid", "contacts": [ { "contact_id": "
  uuid" } ] } }
```

Response 201:

```
{
  "message": "Location successfully shared",
  "data": {
    "trip_id": "uuid",
    "shared_with": [
      {
        "contact_id": "uuid",
        "username": "String"
      }
    ],
    "shared_at": "dateTime"
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 400:

```
{
  "error": "NO_CONTACTS_PROVIDED",
  "message": "Contact list is empty"
}
```

Response 404:

```
{
  "error": "TRIP_NOT_FOUND",
  "message": "Cannot find trip"
}
```

Response 403:

```
{
  "error": "NOT_TRUSTED_CONTACT",
  "message": "Cannot share location with non-trusted contacts"
}
```

Response 404:

```
{
  "error": "USER_NOT_FOUND",
  "message": "Could not find user"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

PATCH /contacts/:contact_id/respond

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{
  "status": "APPROVED | DENIED"
}
```

Response 200:

```
{
  "message": "Status updated successfully",
  "data": {
    "contact_id": "uuid"
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 422:

```
{
  "error": "INVALID_STATUS",
  "message": "Status should be APPROVED or DENIED"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Could not respond to trusted contact request"
}
```

GET /contacts/received_requests

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "data": {
    "requests": [
      {
        "contact_id": "uuid",
        "created_at": "dateTime",
        "username": "String"
      }
    ]
  },
  "message": "Fetched received contact requests"
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Could not get receieved trusted contact requests"
}
```


5.4 Badges Endpoints

POST /badges/evaluate

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{
  "data": {
    "user_id": "uuid (must match authenticated user)",
    "trip_id": "uuid"
  }
}
```

Response 200:

```
{
  "data": {
    "data": {
      "evaluated": "boolean",
      "new_badges": [
        {
          "badge_id": "uuid",
          "name": "string",
          "description": "string",
          "category": "string",
          "earned_at": "dateTime",
          "icon_url": "string"
        }
      ]
    }
  },
  "message": "Badge evaluation complete"
}
```

*Note: the response is double-nested under **data.data** — this reflects the current implementation.*

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 404:

```
{
  "error": "NOT_FOUND",
  "message": "Trip not found"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Could not evaluate badges"
}
```

GET /badges

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "data": {
    "earned": [
      {
        "badge_id": "uuid",
        "name": "string",
        "category": "string",
        "description": "string",
        "current": "int"
      }
    ],
    "summary": {
      "Total_earned": "int",
      "categories": [
        {
          "category": "string",
          "current": "int"
        }
      ]
    }
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Could not retrieve badges"
}
```

GET /badges/definitions

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "data": {
    "badges": [
      {
        "badge_id": "uuid",
        "name": "string",
        "description": "string",
        "category": "string",
        "icon_url": "string",
        "criteria": [
          {
            "metric": "string",
            "operator": "string",
            "threshold": "float | null",
            "target": "float | null"
          }
        ]
      }
    ]
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Could not retrieve badge definitions"
}
```

5.5 Leaderboard Endpoints

GET /leaderboard

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Query Parameters

- category = string (required)
- scope = string (required)

Response 200:

```
{
  "data": {
    "category": "string",
    "scope": "string",
    "entries": [
      {
        "rank": "int",
        "user_id": "uuid",
        "display_name": "String",
        "score": "float"
      }
    ],
    "my_rank": "int | null",
    "my_score": "float"
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 400:

```
{
  "error": "BAD_REQUEST",
  "message": "Missing category or scope query parameters"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Could not retrieve leaderboard"
}
```

GET /leaderboard/categories

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "data": {
    "categories": [
      "string"
    ]
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
}
```

```
{
  "message": "Could not retrieve categories"
}
```

GET /leaderboard/scopes

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "data": {
    "scopes": [
      "string"
    ]
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Could not retrieve categories"
}
```

5.6 Vehicle Endpoints

Note: all vehicle endpoint error responses use a "message" key rather than an "error" key.

GET /vehicle/get_all_vehicles

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
[
  {
    "vehicle_id": "uuid",
    "name": "string | null",
    "registration": "string | null",
    "make": "string",
    "model": "string",
    "year": "int",
    "fuel_type": "string",
    "mileage": "int",
    "trip_count": "int",
    "avg_fuel_efficiency": "float"
  }
]
```

Response 403:

```
{
  "message": "Unauthorized"
}
```

Response 403:

```
{
  "message": "user or vehicle not known"
}
```

Response 403 (user not found):

```
{
  "message": "Unauthorized"
}
```

POST /vehicle/assign_vehicle

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{
  "name": "string (optional)",
  "registration": "string (optional)",
  "make": "string",
}
```

```
"model": "string",  
"year": "number",  
"fuel_type": "string"  
}
```

Response 201:

```
{  
  "data": {  
    "vehicle_id": "uuid",  
    "name": "string | null",  
    "registration": "string | null",  
    "make": "string",  
    "model": "string",  
    "year": "int",  
    "fuel_type": "string"  
  }  
}
```

Response 403:

```
{  
  "message": "Unauthorized"  
}
```

Response 400:

```
{  
  "message": "Missing required fields: make, model, year,  
    fuel_type"  
}
```

Response 404:

```
{  
  "message": "User not found"  
}
```

Response 500:

```
{  
  "message": "Internal server error",  
  "error": "String"  
}
```


PATCH /vehicle/:vehicle_id/name

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{
  "name": "string"
}
```

Response 200:

```
<updated vehicle record: vehicle_id, name, registration, make,
  model, year, fuel_type>
```

Response 401:

```
{
  "message": "Unauthorized"
}
```

Response 500:

```
{
  "message": "Internal server error",
  "error": "String"
}
```

DELETE /vehicle/:vehicle_id

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "message": "Vehicle removed successfully"
}
```

Response 401:

```
{
  "message": "Unauthorized"
}
```

Response 404:

```
{
  "message": "Vehicle not found or not owned by you"
}
```

Response 500:

```
{
  "message": "Internal server error"
}
```

5.7 Map Endpoints

GET /map/token

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "message": "Map token retrieved successfully",
  "data": {
    "token": "string",
    "auth_type": "subscriptionKey"
  }
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED",
  "message": "Can not access map services"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Failed to retrieve map token"
}
```

GET /map/route

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Query Parameters

- start_lat = float
- start_lng = float
- dest_lat = float
- dest_lng = float

Response 200:

```
{
  "message": "Suggested route retrieved",
  "data": {
    "distance_km": "float",
    "travel_time_seconds": "int",
    "traffic_delay_seconds": "int",
    "points": [ { "lat": "float", "lng": "float" }, ... ]
  }
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "String"
}
```

GET /map/search

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Query Parameters

- address = string (required)

Response 201:

```
{
  "message": "name translated to coordinates",
  "data": [
    {
      "address": "string",
      "lat": "float",
      "lng": "float"
    }
  ]
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED",
  "message": "Can not access map services"
}
```

Response 400:

```
{
  "error": "BAD_REQUEST",
  "message": "Address query parameter is required"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Failed to translate address"
}
```

5.8 Notifications Endpoints

GET /notifications

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Response 200:

```
{
  "message": "Notifications retrieved successfully",
}
```

```
"data": {
  "notifications": [
    {
      "notification_id": "uuid",
      "type": "string",
      "title": "string",
      "body": "string | null",
      "reference_id": "uuid | null",
      "reference_type": "string | null",
      "created_at": "dateTime"
    }
  ]
}
```

Response 500 (sic — typo exists in the current implementation):

```
{
  "error": "INTERNAL_SERVER_ERROR",
  "message": "String"
}
```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

5.9 User Devices Endpoints

POST /devices/fcm_token

Headers

```
{
  "Authorization": "Bearer <jwt_token>"
}
```

Request Body

```
{
  "fcm_token": "String"
}
```

Response 201:

```
{
  "message": "Registered successfully"
}
```

```
}

```

Response 401:

```
{
  "error": "UNAUTHORIZED"
}
```

Response 422:

```
{
  "error": "MISSING_FCM_TOKEN",
  "message": "Missing fcm token"
}
```

Response 500:

```
{
  "error": "INTERNAL_SERVER_ERROR"
}
```

5.10 Health Endpoint

GET /health

Response 200:

```
{
  "status": "ok",
  "message": "Driving Tracker API is running"
}
```

5.11 CI pipeline flow diagram

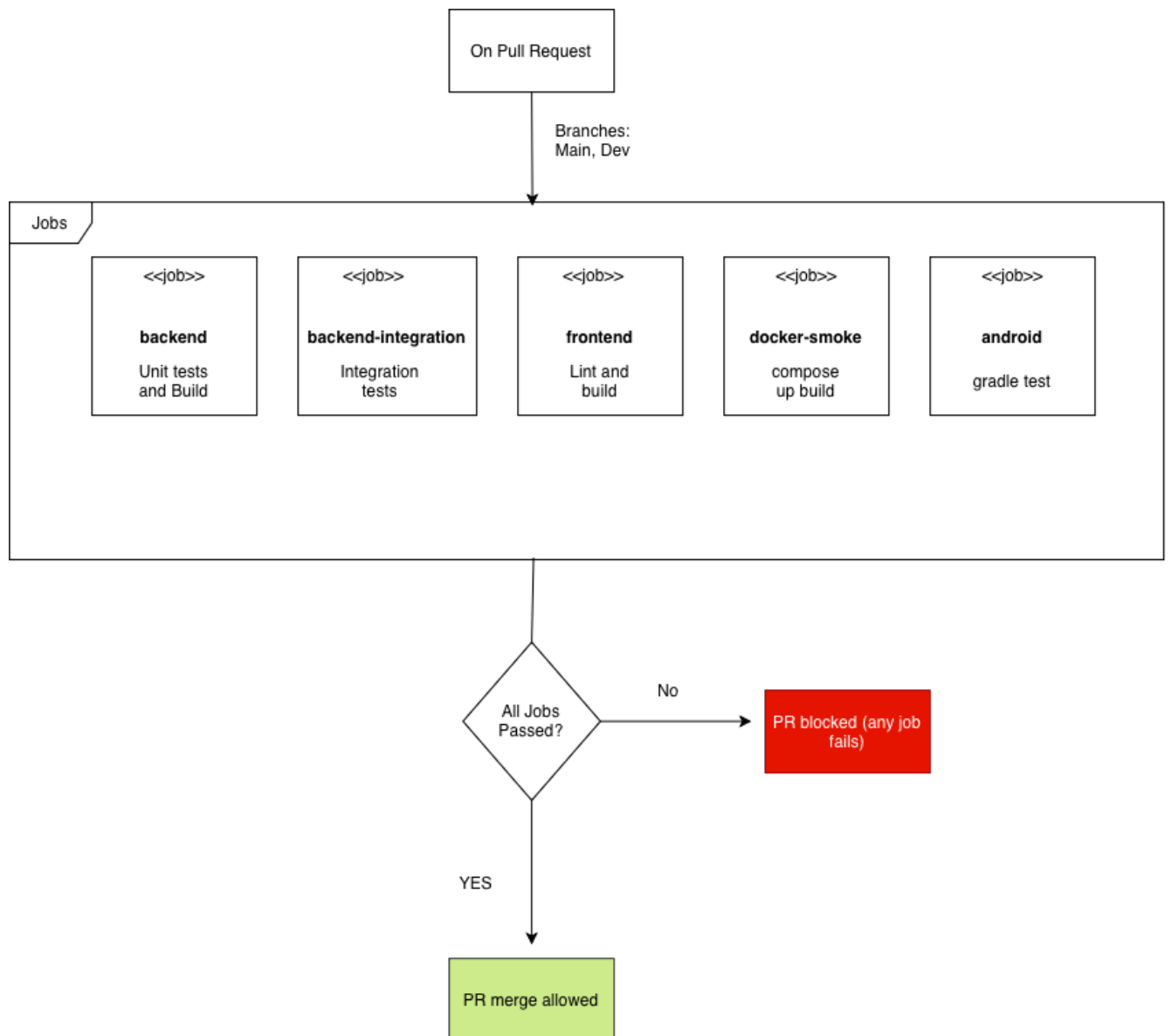


Figure 3: CI pipeline diagram

5.12 CD pipeline flow diagram

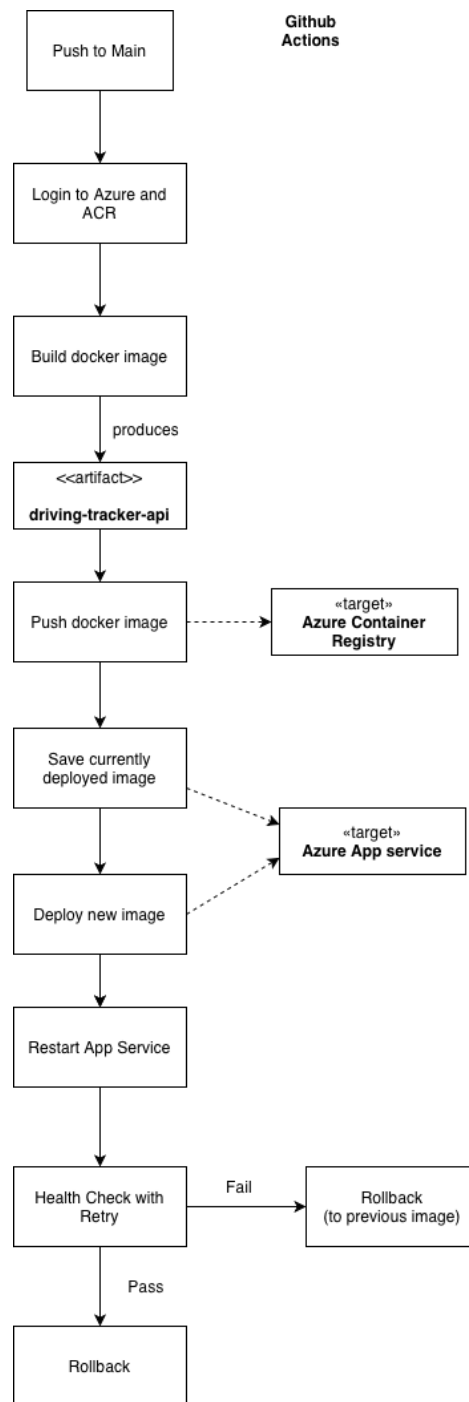


Figure 4: CD pipeline diagram